MTH2008

Scientific computing

Second semester

1st lecture

Danilo Roccatano - Marco Pinna

Office : INB3129

Email: Mpinna@lincoln.ac.uk

Padlet

https://uol.padlet.org/mpinna2/scientific-computing-l5c5cjv5fmvv72l0

Overview Semester B

NUMERICAL METHODS AND ALGORITHMS

- Random Numbers
- Sorting algorithm
- Numerical Differentiation
- Root of a equations
  - Bisection Method
  - Newton Method
  - Secant Method
- Numerical Differentiation
- Numerical Integration
- Numerical integration of Ordinary Differential Equation (first order)
- Linear Algebra
  - Product of scalar with vectors/matrix
  - Product of two matrix

Recap

- (Pseudo) Random generators

1. Mixed congruential methods
2. Multiplicative congruential method

```
Uniform distribution with range [1, 100] - only integers
srand(static_cast<unsigned int>(time(0)));
(rand() % 100) +1;

// Uniform distribuition with range [0,99]
num[c] = ((double)rand() / RAND_MAX )*100;



// Uniform distribution in [0.0, 1.0)
random_device rd;                                   // Random device for seeding
    mt19937 gen(rd());                                  // Mersenne Twister engine
    uniform_real_distribution<double> dist(0.0, 1.0);
    dist(gen);


// for a normal or gaussian distribution
 normal_distribution<> normal_dist_5_1(5.0, 1.0); // Mean = 5, Stddev = 1
```

Overview Semester B

NUMERICAL METHODS AND ALGORITHMS

- Random Numbers
- Sorting algorithm
- Numerical Differentiation
- Root of a equations
  - Bisection Method
  - Newton Method
  - Secant Method
- Numerical Differentiation
- Numerical Integration
- Numerical integration of Ordinary Differential Equation (first order)
- Linear Algebra
  - Product of scalar with vectors/matrix
  - Product of two matrix

LAB task 1.2 - Lotto

a) WRITE A C++ PROGRAM THAT EXTRACT 6 DIFFERENT RANDOM NUMBERS N THE RANGE 1-50 AND PRINT THE LUCKY NUMBERS ON THE SCREEN.

b) EXTEND THE PROGRAM BY EVALUATING THE NUMBER OF EXTRACTIONS REQUIRED TO WIN WITH 2, 3, 4 ,5 and 6 MATCHING NUMBER OUT OF 6 of YOUR CHOICE.

EXTEND THE PROGRAM BY EVALUATING THE NUMBER OF EXTRACTIONS REQUIRED TO WIN WITH 2, 3, 4 ,5 and 6 MATCHING NUMBER OUT OF 6 of YOUR CHOICE.

```cpp
#include <iostream>
#include <iomanip>
#include <random>  // Modern C++ random number library
#include <ctime>

using namespace std;

#define MAXN 50

void mix_and_extract(int lnums[6]);  // Function declaration

void mix_and_extract(int lnums[6]) {
    int i, j, k, nums[MAXN];

    // Fill nums[] with numbers from 1 to 50
    for (i = 0; i < MAXN; i++) nums[i] = i + 1;

    // Create a random number generator using Mersenne Twister
    random_device rd;
    mt19937 gen(rd());  // Seeded Mersenne Twister
    uniform_int_distribution<int> dist(0, MAXN - 1);  // Uniform distribution from 0 to 49

    // Shuffle nums[] using original swapping method
    for (i = 0; i < MAXN; i++) {
        j = dist(gen);  // Generates a random index in range 0-49
        k = nums[i];
        nums[i] = nums[j];  // Swap nums[i] with nums[j]
        nums[j] = k;
    }

  // Extract the first 6 numbers from nums[]
    for (i = 0; i < 6; i++) lnums[i] = nums[i];
}
```

```
int main() {
    int m, i, j, k, val, chv[MAXN+1] = {0}, lnums[6], ynums[6];
    int nextr = 0, Counter[7] = {0};

    srand((int) time(0));  // Seed the random number generator

    cout << "******** SCIENTIFIC COMPUTING LOTTERY *********\n";
    cout << "********          TRY YOUR LUCK          *********\n";
    cout << "********       BY GIVING 6 NUMBERS       *********\n";

    // User input: Choose 6 numbers (ensures no duplicates)
    for (i = 0; i < 6; i++) {
        do {
            cout << "N. " << (i + 1) << "? ";
            cin >> val;

            if (val < 1 || val > 50)
                cout << "Only numbers between 1-50, please!\n";

            if (chv[val] == 1)
                cout << "This number was already selected!\n";

        } while ((val < 1 || val > 50) || chv[val] != 0);

        ynums[i] = val;  // Store valid number
        chv[val] = 1;    // Mark number as used
    }
```

```
    cout << "THANK YOU! \n";
    cout << "SHALL NOW TURN THE WHEEL OF FORTUNE ... \n";

    // Lottery drawing and matching process
    do {
        mix_and_extract(lnums);
        m = 0;

        for (i = 0; i < 6; i++) {  // Check for matches
            for (j = 0; j < 6; j++) {
                if (ynums[j] == lnums[i]) {
                    m++;
                }
            }
        }
//counters how many times you generate a random number
        if (m != 0) {
            if (Counter[m] == 0) {
                for (k = 1; k <= m; k++) {
                    if (Counter[k] == 0) Counter[k] = nextr;
                }
            }
        }
        nextr++;

    } while (m < 6);
```

```
    // Final output displaying match statistics
    cout << setprecision(9) << fixed;
    cout << "\nN. winning numbers | Number of draws | Probability | Weeks " << endl;
    cout <<   "-------------------|-----------------|-------------|-------" << endl;
    for (i = 1; i < 7; i++) {
        cout << i << "                    | " << setw(16) << Counter[i]
             << " | " << (1.0 / Counter[i]) << " | "
             << Counter[i] / 2 << endl;
    }

    return 0;
}
```

Simple sorting algorithm

Bubble Sort Algorithm (1 of 6)

- `list[0]...list[n - 1]`
  - List of *n* elements, indexed 0 to *n* – 1
  - Example: a list of five elements (Figure 1)

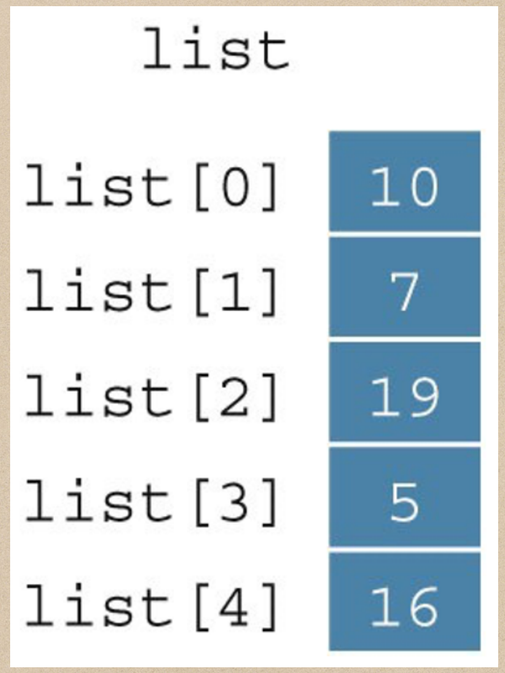


Figure 1 - Initial list. [1]

[1] Cengage learning

Bubble Sort Algorithm (2 of 6)

- Series of `n - 1` iterations
  - Successive elements `list[index]` and `list[index + 1]` of `list` are compared
  - If `list[index] > list[index + 1]`
    - Swap `list[index]` and `list[index + 1]`
  - Smaller elements move toward the top (beginning of the list)
  - Larger elements move toward the bottom (end of the list)

Bubble Sort Algorithm

(3 of 6)

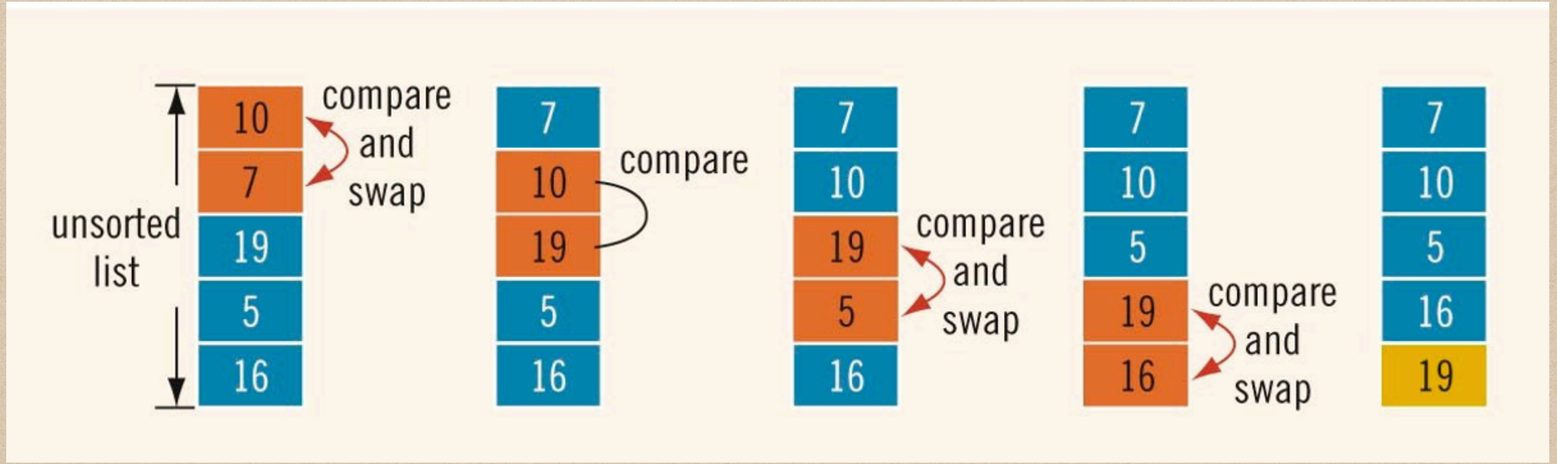


Fig - 1.1 1 iteration [1]

[1] Cengage Learning

Bubble Sort Algorithm (4 of 6)

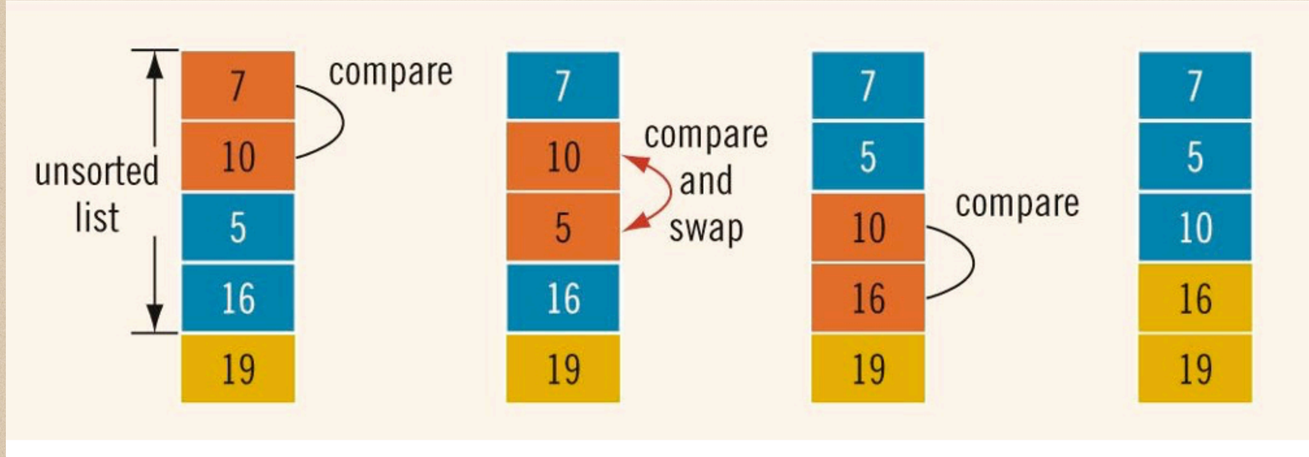


Fig - 1.2 - 2nd iteration [1]

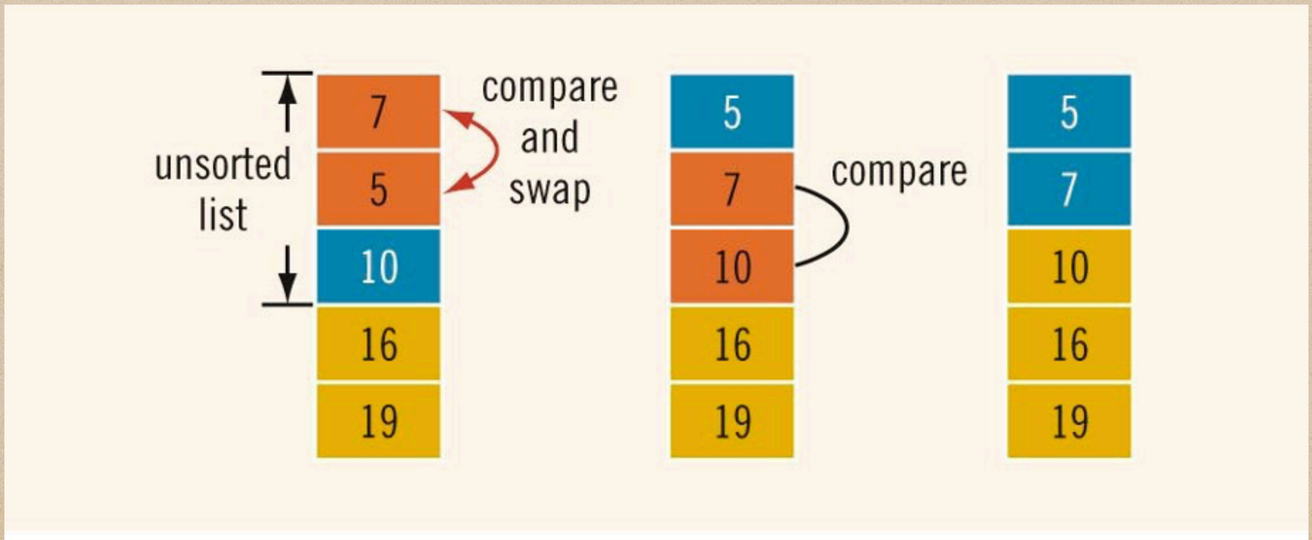


Fig - 1.3 - 3rd iteration [1]

[1] Cengage Learning

Bubble Sort Algorithm

(5 of 6)

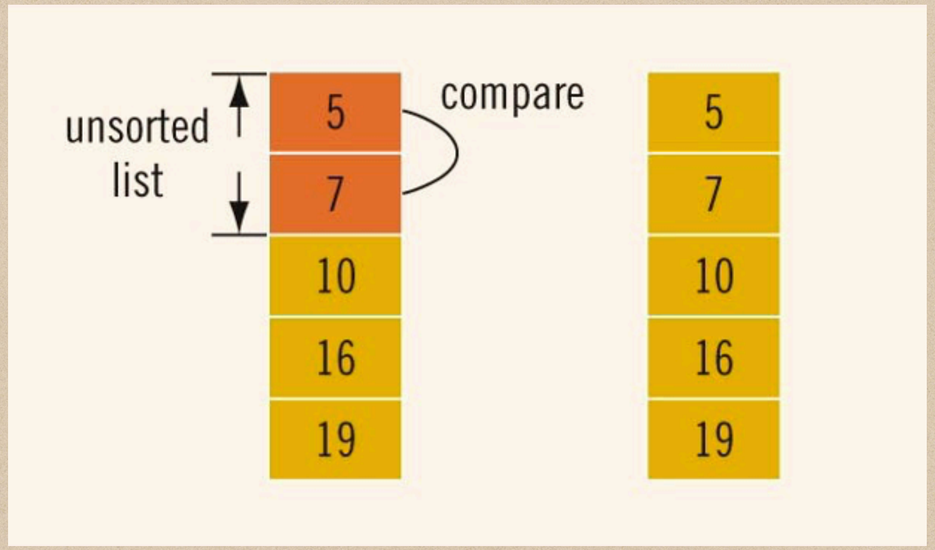


Fig - 1.3 - 4th iteration [1]

[1] Cengage Learning

Bubble Sort Algorithm (6 of 6)

- List of length *n*
  - Exactly *n*(*n* - 1) / 2 key comparisons
  - On average *n*(*n* - 1) / 4 item assignments
- If *n* = 1000
  - 500,000 key comparisons and 250,000 item assignments
- Can improve performance if we stop the sort when no swapping occurs in an iteration

```cpp
#include <iostream>
#include <cstdlib>  // For rand() and srand()
#include <ctime>    // For time()
using namespace std;

int main() {
    int a, b;
    int num[10];  // Declare an array of 10 numbers
    int size = 10;  // Define the number of elements

    srand(time(0));  // Seed the random generator to avoid repetitive sequences

    // Initialize the array with random numbers between 1 and 100
    for (int c = 0; c < size; c++) {
        num[c] = rand() % 100 + 1;  // Random numbers between 1 and 100
    }

    // Display original numbers
    cout << "Original numbers: ";
    for (int c = 0; c < size; c++) {
        cout << num[c] << " ";
    }
    cout << endl;

    // Optimized Bubble Sort with early stopping
    bool swapped;
    for (a = 0; a < size - 1; a++) {
        swapped = false;  // Reset swap flag
        for (b = 0; b < size - 1 - a; b++) {  // Inner loop shrinks as elements get sorted
            if (num[b] > num[b + 1]) {
                swap(num[b], num[b + 1]);  // Use swap function
                swapped = true;
            }
        }
        if (!swapped) break;  // If no swaps, array is already sorted
    }

    // Display sorted numbers
    cout << "Sorted numbers: ";
    for (int c = 0; c < size; c++) {
        cout << num[c] << " ";
    }
    cout << endl;

    return 0;
}
```

Bubble Sort Algorithm

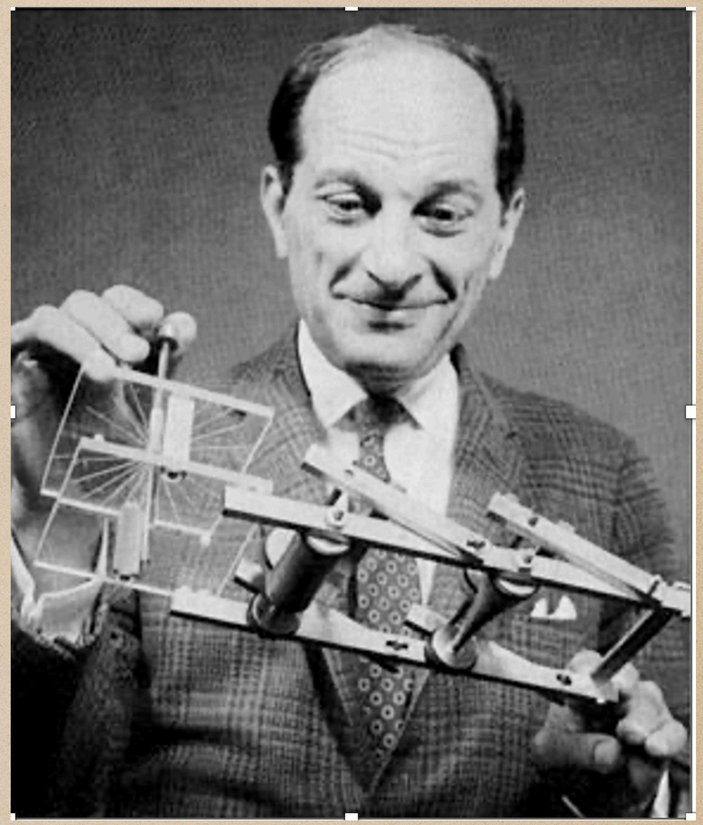
Output

```
Original numbers: 34 80 80 1 19 22 32 15 51 51
Sorted numbers: 1 15 19 22 32 34 51 51 80 80
Total iterations: 42
```

Applications of random Numbers

Monte Carlo Simulations

Monte Carlo simulation was pioneered during the Manhattan project by Stanislaw Ulam (1909- 1984) and others, who recognized that this scheme permitted simulations beyond the reach of conventional methods on the limited computer systems then available.



Metropolis N., Ulam S. The Monte Carlo method, *J. Amer. Statistical Assoc.*, 1949, 44, No. 247, 335-341.

Monte Carlo Simulations

Monte Carlo simulations have great importance in many areas of science. This a small list of applications:

- Evaluating integrals of arbitrary functions of 6+ dimensions
- Predicting future values of stocks
- Solving partial differential equations
- Sharpening satellite images
- Modeling cell populations
- Finding approximate solutions to NP-hard problems

Monte Carlo Simulations - Calculation of the logarithm using Monte Carlo

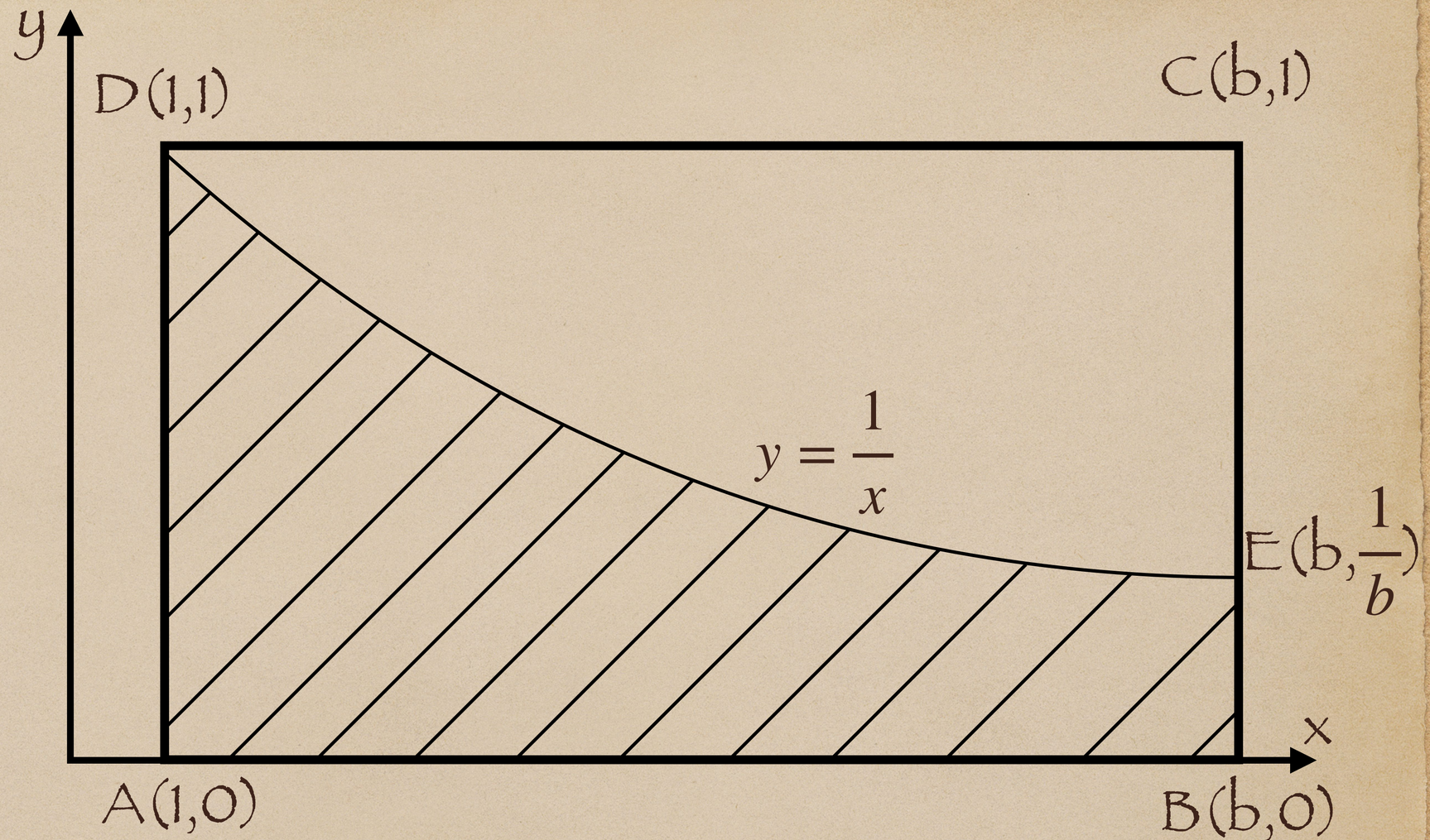
Shall we consider the function y = ln(x) in the interval [1, b] with b> 1.

Let ABCD be the rectangle of vertices A = (1,0); B = (b, 0); C = {b, 1); D = (1, 1) and ABED the trapezoid bounded by the x axis, by the 1/x graph and by the lines x = 1 and x = b.

The point E is given by (b, 1 /b) - see figure).

The probability that a point P, chosen at random in the rectangle, also belongs to the trapezoid is given by the ratio of the areas of the two figures:

p = S (trapezoid ABED) / S (rectangle ABCD).



Calculation of the logarithm using Monte Carlo

The probability that a point P, chosen at random in the rectangle, also belongs to the trapezoid is given by the ratio of the areas of the two figures:

p = S (*trapezoid* ABED) / S (*rectangle* ABCD).

(Eq. 1)

Therefore,

$$S(trapezoid) = \int_1^b \frac{1}{x}\,dx = ln(b).$$ **(Eq. 2)**

and

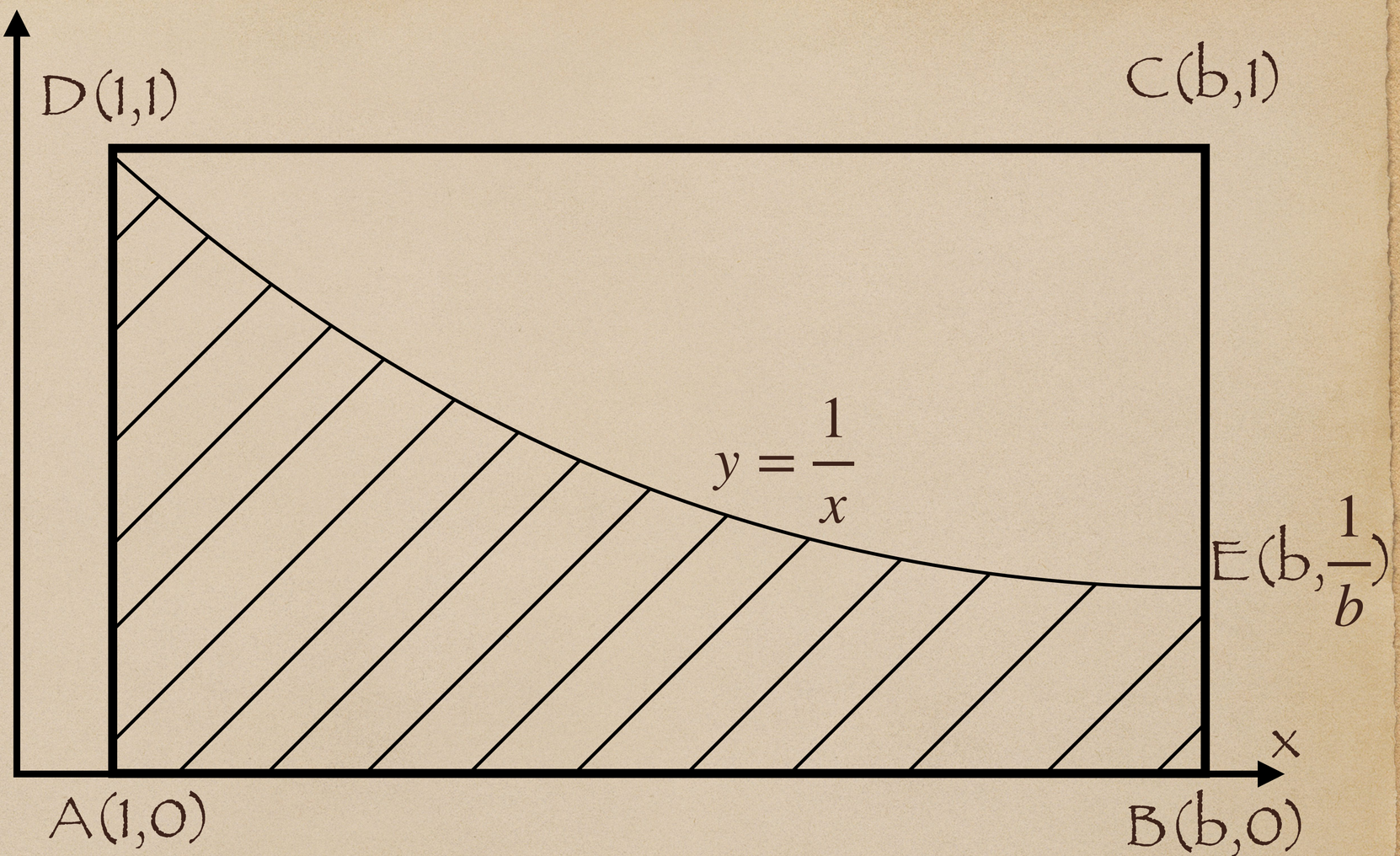
$$S(rectangle) = (b-1).$$ **(Eq. 3)**

Hence, by combining Eqs 2 and 3 in Eq. 1, we obtain

$$p = \frac{ln(b)}{b-1}$$ **(Eq. 4)**

and, therefore,

$$ln(b) = p \cdot (b-1)$$ **(Eq. 5)**



Monte Carlo Simulations: Exercise task 2.1

Write a program that evaluates the integral by using Monte Carlo approach

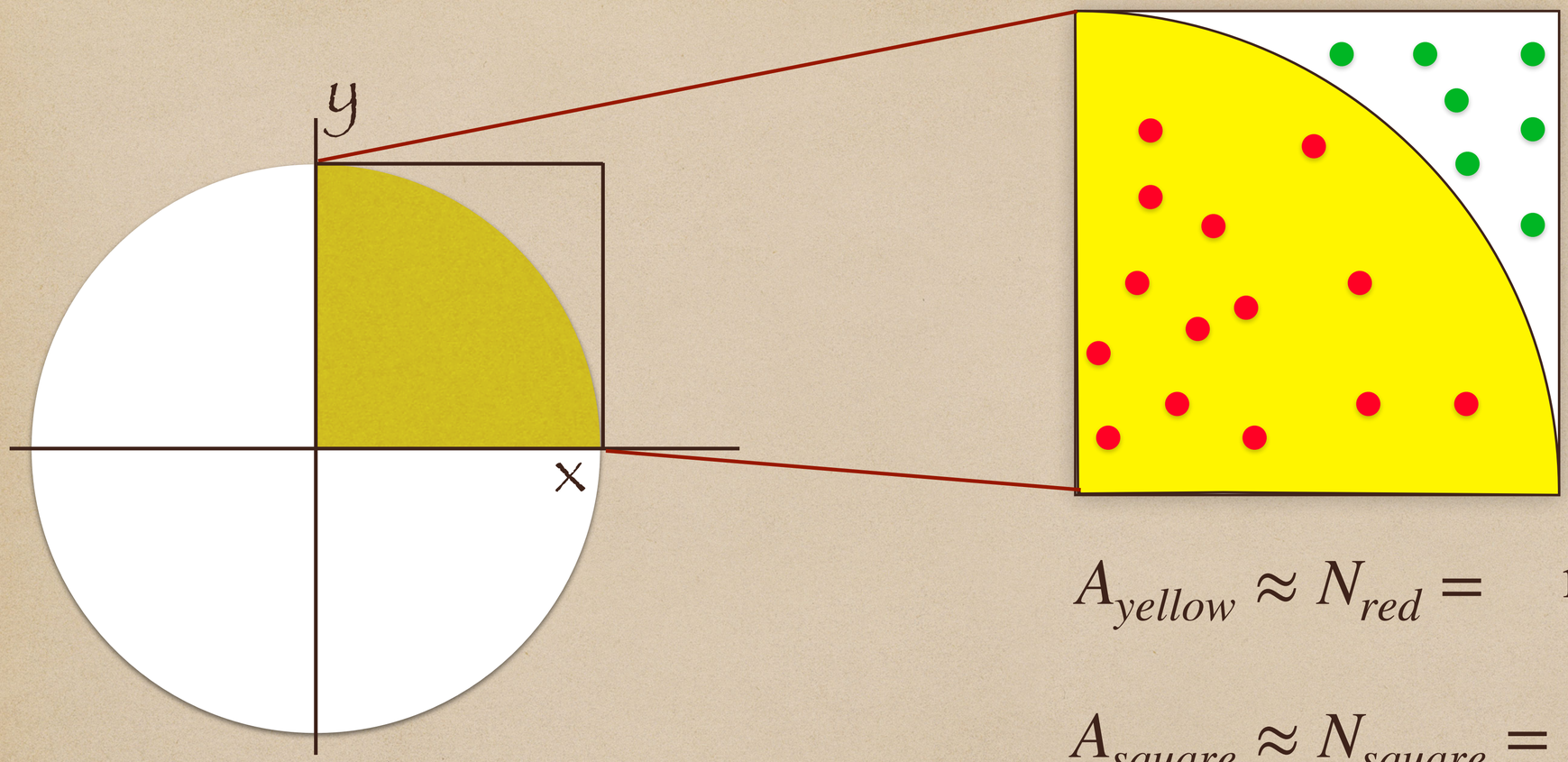
Algorithm

- Insert the value b of which the natural logarithm is requested.
- Generate a uniform random number U in [0,1) for the Y
- Generate a random number between 1 and b for X coordinate.
- Check if the coordinates (X,Y) are under the curve 1/X. If so, increment the variable S(trapezoid) of one unit.
- Repeat for a given large number of steps (N=number of iterations).
- Calculate the logarithm as AL=S*(b-1)/N
- Evaluate the error by comparing with the C++ function for the natural logarithm.

Monte Carlo Simulations: Task 2.2

- Write a c++ function that calculates the value of π with a given accuracy using the Monte Carlo Method.
- Evaluate the convergence as function of N (number of points)

Monte Carlo Simulations: Task 2.2



$$A_{yellow} \approx N_{red} = \quad \text{n points in red}$$

$$A_{square} \approx N_{square} = \quad \text{total n points}$$

$$A_{yellow} = \frac{1}{4}\pi r^2$$

$$A_{square} = r^2$$

$$\frac{A_{yellow}}{A_{square}} = \frac{1}{4}\pi$$

$$\pi = 4 \cdot \frac{A_{yellow}}{A_{square}}$$

Questions?